

DOCUMENTO TÉCNICO

Validación y pruebas

Pruebas funcionales, casos extremos, integración y resultados del contenedor

Campo	Detalle
Proyecto	Clasificación de cáncer de mama - Breast Cancer Wisconsin Diagnostic
Repositorio	https://github.com/HamDan314/cancermm
Responsable	Leonel Cardoso Gómez
Fecha	2 de agosto de 2026
Carácter	Proyecto académico; no constituye diagnóstico médico

1. Propósito

El propósito es comprobar la carga del modelo, el contrato de la API, la integración frontend-backend, el rechazo de entradas inválidas y la construcción reproducible de imágenes Docker.

2. Alcance

- Carga del pipeline de scikit-learn y metadatos versionados.
- Endpoints /, /health, /model-info y /predict.
- Validación de 30 características numéricas.
- Probabilidades y consistencia de la respuesta.
- Proxy Nginx /api hacia el backend.
- Pruebas automatizadas y construcción Docker en GitHub Actions.

3. Entorno de validación

Elemento	Configuración
Python	3.11 en GitHub Actions; validación local adicional con Python 3.12.
FastAPI	0.115.12
scikit-learn	1.8.0, coincidente con la serialización del modelo.
Pytest	8.3.5
Contenedores	python:3.11-slim y nginx:1.27-alpine
Repositorio	HamDan314/cancermm, rama main

4. Criterios de aceptación

1. Las siete pruebas automatizadas deben aprobar.
2. /health debe responder HTTP 200 y model_loaded: true.
3. Una solicitud válida debe devolver diagnóstico y dos probabilidades entre 0 y 1.
4. La suma de probabilidades debe ser aproximadamente 1.
5. Las entradas incompletas, adicionales o no numéricas deben devolver HTTP 422.
6. GitHub Actions debe construir las imágenes del backend y frontend sin errores.

5. Casos funcionales

ID	Prueba	Entrada	Resultado esperado
CP-01	Salud	GET /health	HTTP 200 y modelo cargado.
CP-02	Predicción válida	30 números	Clase, probabilidades, latencia y versión.
CP-03	Información	GET /model-info	Variables, versión y métricas.
CP-04	Integración web	Formulario → /api/predict	Resultado visible en el frontend.
CP-05	Proxy	GET /api/health en 8080	HTTP 200 reenviado al backend.
CP-06	Documentación API	GET /docs	Swagger disponible.

6. Casos extremos

ID	Caso	Resultado esperado
CE-01	Menos de 30 variables	HTTP 422.
CE-02	Más de 30 variables	HTTP 422.
CE-03	Valores no numéricos	HTTP 422.
CE-04	NaN o infinito	HTTP 422.
CE-05	Cuerpo ausente	HTTP 422.
CE-06	Campo inesperado	HTTP 422 por configuración extra=forbid.
CE-07	Modelo o metadatos ausentes	El servicio no inicia o responde 503.

7. Resultados automatizados

Resultado local registrado: 7 passed. El Pull Request #1 ejecutó GitHub Actions después de corregir la invocación a `python -m pytest -q`.

Comprobación	Resultado	Duración observada
CI/test (push)	Successful	55 s
CI/test (pull_request)	Successful	55 s
Pruebas Pytest	7 aprobadas	Incluidas en el job CI/test
Imagen backend	Construcción correcta	Incluida en el job CI/test
Imagen frontend	Construcción correcta	Incluida en el job CI/test

8. Métricas del modelo desplegado

Métrica	Resultado
Accuracy	0.9825
Precision	0.9861
Recall	0.9861
F1-score	0.9861
ROC-AUC	0.9954
Muestras de prueba	114

9. Evidencia de integración

Evidencia	Ubicación
Código de endpoints	backend/app/main.py
Pruebas automatizadas	backend/tests/test_api.py
Orquestación	docker-compose.yml
Proxy frontend-backend	frontend/nginx.conf
Workflow CI	.github/workflows/ci.yml
Historial verificable	https://github.com/HamDan314/cancermm/pull/1

10. Interpretación de resultados

La aprobación de Pytest confirma el contrato principal de la API y el manejo de entradas inválidas. La construcción exitosa de las dos imágenes demuestra que los Dockerfiles contienen dependencias, rutas y comandos de inicio coherentes. La ejecución en eventos push y pull_request aporta evidencia independiente y reproducible en GitHub.

11. Limitaciones

- El dataset es académico y no representa por sí solo una validación clínica prospectiva.
- Las pruebas automatizadas no sustituyen pruebas de carga, seguridad o penetración.
- La salida del modelo no constituye diagnóstico médico.
- Para datos reales se requieren controles de privacidad, autenticación, auditoría y supervisión clínica.

12. Conclusiones

La solución satisface los criterios funcionales definidos: carga el modelo, valida 30 características, expone endpoints observables, integra frontend y backend, rechaza entradas inválidas y construye imágenes reproducibles. Los dos controles exitosos de GitHub Actions constituyen la evidencia principal de validación continua.